# Container Security: From Basics to Advanced Protection

## Complete Course Outline

**Total Duration:** 3 hours 30 minutes - 4 hours
**Level:** Intermediate
**Prerequisites:** Basic Docker/container knowledge, Linux command line familiarity

---

## Module 0: Course Introduction

**Duration:** 15-20 minutes

### Videos:

1. **Welcome and Course Overview** (8-10 mins)
   - Course structure and learning path
   - Target audience and prerequisites
   - Lab environment setup overview
   - What you'll build by the end

2. **Container Security Landscape** (7-10 mins)
   - Why container security matters
   - Common misconceptions about container isolation
   - Real-world breach examples
   - Security layers overview

### Learning Objectives:

- Understand the scope and structure of the course
- Recognize the importance of container security
- Set up the lab environment successfully
- Identify key security challenges in containerized environments

---

## Module 1: Container Security Fundamentals

**Duration:** 30-35 minutes

### Videos:

1. **Container Isolation Deep Dive** (10-12 mins)
   - How containers actually work (namespaces, cgroups)
   - The shared kernel reality
   - Isolation vs. virtualization
   - **Demo:** Examining namespace isolation with live commands

2. **Privileged Containers: The Security Nightmare** (10-12 mins)
   - What makes a container privileged
   - Capabilities and their risks
   - **Lab:** Creating and examining privileged containers
   - **Demo:** Breaking out of a privileged container

3. **Container APIs and Attack Surface** (10-11 mins)
   - Docker daemon socket exposure
   - Remote API vulnerabilities
   - **Demo:** Exploiting exposed Docker socket
   - Securing container APIs

### Learning Objectives:

- Explain how Linux namespaces and cgroups provide container isolation
- Identify the risks of privileged containers
- Recognize exposed container APIs as critical vulnerabilities
- Demonstrate a privileged container escape attack

### Hands-On Labs:

- Lab 1.1: Exploring namespace isolation
- Lab 1.2: Privileged container exploitation
- Lab 1.3: Docker socket vulnerability demonstration

---

## Module 2: Container Attack Vectors and Vulnerabilities

**Duration:** 35-40 minutes

### Videos:

1. **Container Escape Techniques** (12-15 mins)
   - Kernel exploits through containers
   - Capability abuse scenarios
   - **Demo:** CVE-based container escape (using safe examples)
   - Detection and prevention strategies

2. **Image Security and Tampering** (12-13 mins)
   - Malicious image analysis
   - Supply chain attacks on images
   - **Lab:** Analyzing a compromised image
   - Image signing and verification with Docker Content Trust
   - **Demo:** Setting up content trust

3. **Insecure Configurations and DoS Attacks** (11-12 mins)
   - Common misconfigurations (volume mounts, network modes)
   - Resource limit bypasses
   - **Demo:** Fork bomb and resource exhaustion attacks
   - Implementing resource constraints
   - **Lab:** Configuring secure container limits

### Learning Objectives:

- Identify and exploit common container escape vectors
- Analyze container images for security vulnerabilities
- Recognize insecure container configurations
- Implement resource controls to prevent DoS attacks
- Use Docker Content Trust for image verification

### Hands-On Labs:

- Lab 2.1: Container escape simulation
- Lab 2.2: Compromised image forensics
- Lab 2.3: Resource limit configuration and testing

---

## Module 3: Seccomp Profiles in Practice ⭐ (Sample Module)

**Duration:** 40-45 minutes

### Videos:

1. **Introduction to Seccomp** (12-14 mins)
   - What is Seccomp and why it matters
   - System call filtering fundamentals
   - Default Docker Seccomp profile analysis
   - **Demo:** Viewing active Seccomp profiles
   - **Lab:** Testing default profile restrictions

2. **Creating Custom Seccomp Profiles** (14-16 mins)
   - Seccomp profile syntax and structure
   - Identifying required syscalls for applications
   - **Live Coding:** Building a profile from scratch
   - **Demo:** Using strace to discover syscall requirements
   - Testing and validating profiles
   - **Lab:** Create profiles for nginx and Python apps

3. **Advanced Seccomp: Troubleshooting and Best Practices** (14-15 mins)
   - Debugging Seccomp denials
   - **Demo:** Reading audit logs and kernel messages
   - Balancing security and functionality
   - Profile maintenance strategies
   - **Lab:** Troubleshooting a broken application
   - Real-world profile examples
   - Integration with orchestration platforms

### Learning Objectives:

- Explain how Seccomp filters system calls at the kernel level
- Analyze and interpret the default Docker Seccomp profile
- Use strace and audit tools to identify application syscall requirements
- Create custom Seccomp profiles for specific applications

- Debug and troubleshoot Seccomp-related application failures
- Apply Seccomp profiles in production environments

### Hands-On Labs:

- Lab 3.1: Exploring default Seccomp behavior
- Lab 3.2: Building custom profiles with strace
- Lab 3.3: Multi-container application profiling
- Lab 3.4: Troubleshooting and fixing profile issues

---

## Module 4: Mandatory Access Control - AppArmor and SELinux

**Duration:** 30-35 minutes

### Videos:

1. **AppArmor Fundamentals** (14-16 mins)
   - MAC vs. DAC security models
   - AppArmor architecture and modes
   - Default Docker AppArmor profile
   - **Demo:** Examining AppArmor enforcement
   - **Live Coding:** Creating custom AppArmor profiles
   - **Lab:** Profile generation and refinement

2. **SELinux for Container Security** (16-19 mins)
   - SELinux context and labels
   - Container-specific SELinux policies
   - **Demo:** SELinux in enforcing mode with containers
   - Multi-category security (MCS) for isolation
   - **Lab:** Configuring SELinux contexts
   - **Demo:** Troubleshooting SELinux denials
   - Choosing between AppArmor and SELinux

### Learning Objectives:

- Differentiate between DAC and MAC security models
- Configure and enforce AppArmor profiles for containers
- Understand SELinux contexts and their role in container isolation
- Generate and customize MAC profiles for applications
- Troubleshoot AppArmor and SELinux policy violations
- Select the appropriate MAC system for your environment

### Hands-On Labs:

- Lab 4.1: AppArmor profile creation workflow
- Lab 4.2: SELinux context management
- Lab 4.3: Comparative analysis - AppArmor vs SELinux

---

# Module 5: Orchestration and Supply Chain Security

**Duration:** 30-35 minutes

## Videos:

1. **Kubernetes Security Fundamentals** (14-16 mins)
   - Pod Security Standards (Restricted, Baseline, Privileged)
   - Security contexts and policies
   - **Demo:** Implementing Pod Security Admission
   - Network policies for container isolation
   - **Lab:** Creating restrictive security contexts
   - **Demo:** Network policy enforcement

2. **Supply Chain Security and Best Practices** (16-19 mins)
   - Image scanning and vulnerability management
   - **Demo:** Trivy and Grype in action
   - Software Bill of Materials (SBOM)
   - Signed artifacts with Sigstore/Cosign
   - **Lab:** Setting up automated image scanning
   - **Demo:** Image signing workflow
   - Runtime security monitoring
   - **Demo:** Falco for runtime threat detection

## Learning Objectives:

- Implement Pod Security Standards in Kubernetes
- Configure security contexts for least privilege access
- Use network policies to enforce container isolation
- Integrate image scanning into CI/CD pipelines
- Sign and verify container images
- Deploy runtime security monitoring tools
- Develop a comprehensive container security strategy

## Hands-On Labs:

- Lab 5.1: Pod Security Standards implementation
- Lab 5.2: Image scanning pipeline setup
- Lab 5.3: Runtime monitoring with Falco

---

# Module 6: Conclusion and Real-World Application

**Duration:** 10-15 minutes

## Videos:

1. **Security Layering Strategy** (5-7 mins)
   - Defense in depth for containers
   - Combining Seccomp, AppArmor/SELinux, and policies
   - Risk assessment framework
   - **Visual:** Security maturity model

2. **Next Steps and Resources** (5-8 mins)
   - Implementing security in existing environments
   - Staying current with container security
   - Community resources and tools
   - Common pitfalls to avoid
   - Final Q&A topics

### Learning Objectives:

- Design a layered container security architecture
- Prioritize security controls based on risk assessment
- Create an implementation roadmap for your organization
- Access ongoing learning resources

---

## Course Delivery Notes

### Content Mix (per publisher requirements):

- **PowerPoint/Slides:** 20-25% (concepts, architecture diagrams, security models)
- **Terminal/CLI:** 35-40% (live demos, command execution, log analysis)
- **Code Editor:** 20-25% (profile creation, YAML files, configuration)
- **Browser:** 15-20% (documentation, dashboards, scanning tools)

### Technical Standards:

- Resolution: 1920x1080 (Full HD)
- Frame Rate: 30 FPS constant
- Screen Zoom: 150% for all applications
- Code Editor: Font size adjusted to show 20-22 lines maximum
- Terminal: Large font with high contrast theme
- All windows maximized

### Pedagogical Approach:

- **No lecture-only content** - every concept paired with demonstration
- **Progressive complexity** - each module builds on previous knowledge
- **Immediate practice** - hands-on labs follow theory within same video
- **Real-world scenarios** - all examples based on actual vulnerabilities
- **Troubleshooting focus** - teach debugging alongside implementation

### Assessment Strategy:

- **Module-level quizzes:** 3 MCQs per module (18 total)
- **Hands-on labs:** 12 practical exercises with validation scripts
- **Final assessment:** 10 comprehensive MCQs + 1 capstone lab
- **Passing criteria:** 75% on assessments + all labs completed

---

# Equipment and Software Requirements

## Instructor Setup:

- Linux environment (Ubuntu 20.04+ or similar)
- Docker Engine 24.0+
- Kubernetes cluster (minikube/kind acceptable)
- Code editor (VS Code recommended)
- Screen recording: OBS Studio or similar (1920x1080, 30fps)

## Student Lab Environment:

- 4GB RAM minimum, 8GB recommended
- 20GB free disk space
- Linux VM or WSL2 on Windows
- Docker Engine and Docker Compose
- kubectl CLI
- Basic tools: strace, auditd, jq, curl

## Pre-configured Lab Resources:

- Vulnerable container images (intentionally insecure)
- Sample Seccomp, AppArmor, and SELinux profiles
- Kubernetes manifests for all demos
- Validation scripts for lab exercises
- Reference documentation bundle

---

# Version History

- **v1.0** - Initial Phase 1 outline
- **Target:** 3.5-4 hour complete course
- **Focus:** Practical, hands-on container security with emphasis on Seccomp, AppArmor, and SELinux